QueryAPIGate HOW-TO GUIDE 17 · ACCESS

Rate-limit a key, or the whole server

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=17-rate-limit-a-key

Time: 10 minutes. **You'll end up with:** a quota on how often callers may hit your API - per API key, for the whole server, or both - and an understanding of the headers and errors a client sees.

There are two independent limits. Both are off until you set them, and when both are set a request must pass both:

	SERVER-WIDE	PER KEY
Set by	`QUERYAPIGATE_RATE_LIMIT` (environment)	the key's own `rate_limit` field
Counted per	client IP address	API key
Applies to	every request except `/health`, `/metrics` and the Console page - the admin key included	that key's requests only
Headers	`X-RateLimit-Limit`, `X-RateLimit-Remaining`	`X-RateLimit-Key-Limit`, `X-RateLimit-Key-Remaining`
429 message	`Rate limit exceeded`	`Rate limit exceeded for this API key`

Both use the same grammar - a count, a slash, and `second`, `minute`, `hour` or `day` (`60/minute`, `5000/day`) - and the same algorithm: a token bucket that holds the full quota and refills continuously. A client may burst up to the quota at once; after that it gets one request per `period / count`.

A limit for one key

Set `rate_limit` when creating the key, or `PATCH` it onto an existing one:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type: application/json' -d '{
  "name": "acme-limited", "connections": [], "queries": ["orders_by_status"], "rate_limit": "3/minute"
}'
```

Four calls in a row, verified:

```shell
curl -i 'http://127.0.0.1:5000/q/orders_by_status?status=paid' -H 'X-API-Key: <acme-limited secret>'
# 1-3:  HTTP/1.1 200 OK
#       X-RateLimit-Key-Limit: 3
#       X-RateLimit-Key-Remaining: 2, then 1, then 0
# 4:    HTTP/1.1 429 TOO MANY REQUESTS
#       Retry-After: 20
#       {"error": "Rate limit exceeded for this API key", "code": "rate_limited", "retry_after": 20,
...}
```

`Retry-After` (and `retry_after` in the body) is the number of seconds until the next request will succeed - 20 here, because a `3/minute` bucket regains one request every 20 seconds. A well-behaved client waits that long rather than retrying in a loop.

Change it, or remove it, without rotating the secret:

```shell
curl -X PATCH http://127.0.0.1:5000/api/v1/api-keys/acme-limited -H 'X-API-Key: demo-key' -H 'Content-
Type: application/json' \
  -d '{"rate_limit": "100/minute"}'
curl -X PATCH http://127.0.0.1:5000/api/v1/api-keys/acme-limited -H 'X-API-Key: demo-key' -H 'Content-
Type: application/json' \
  -d '{"rate_limit": null}'
```

A malformed value is refused when it's saved, not discovered later: `"3/fortnight"` gets `400` - *rate_limit must look like '60/minute' (a positive count, then second, minute, hour or day).*

A key can read its own limit from `GET /catalog` (`caller.rate_limit`, and `caller.server_rate_limit` for the server-wide one), so a partner's client can pace itself without asking you.

A limit for the whole server

```shell
QUERYAPIGATE_API_KEY=demo-key QUERYAPIGATE_RATE_LIMIT=600/minute queryapigate serve
```

Every response now carries `X-RateLimit-Limit` and `X-RateLimit-Remaining`; a key with its own limit gets both pairs. This limit is checked **before** the API key, so it also throttles someone guessing keys. It counts by IP address, so it includes the admin key and the Console's own requests - set it with room for those.

Behind a reverse proxy: `QUERYAPIGATE_TRUST_PROXY`

Behind nginx, Caddy or a load balancer, every request reaches QueryAPIGate from the proxy's address. Without telling it otherwise, all your callers share **one** server-wide bucket. Set `QUERYAPIGATE_TRUST_PROXY` to the number of proxies in front of it, and the client address is taken from `X-Forwarded-For` instead.

Verified with `QUERYAPIGATE_RATE_LIMIT=2/minute`, three requests "from" `198.51.100.7`, then one from `203.0.113.9`:

	198.51.100.7 ×3	203.0.113.9
`QUERYAPIGATE_TRUST_PROXY` unset	200, 200, 429	**429** - same bucket: the header is ignored
`QUERYAPIGATE_TRUST_PROXY=1`	200, 200, 429	**200** - its own bucket

Only set it when there really is a proxy that overwrites `X-Forwarded-For`. Without one, any caller could send a new `X-Forwarded-For` value with each request and never be limited. The same client address is what `allowed_ips` checks, so this setting matters for both. See Put QueryAPIGate behind a reverse proxy.

Seeing who hits their limits

- **Alerts** (Console, under **Observability**) calls out a key - or, for the server-wide limit, a client address - refused 10 or more times in the last hour, with what to do about it.
- `/metrics` counts every refusal in `queryapigate_rate_limit_rejections_total`; the Metrics screen shows it too.

Limits of the limits

Without Redis, the counters live in each server process's memory: they reset on restart, and several instances behind a load balancer each count separately, so the effective limit is multiplied by the number of instances.

With `QUERYAPIGATE_REDIS_URL` set, the limits are shared: every instance, and `queryapigate mcp` and `events`, counts against one budget per client address and per key, kept in Redis. If Redis goes down, each process counts on its own until it's back - limits stay enforced, just per process - and **Alerts** shows *Rate limits are counted per instance*. Behind the Rust data plane (guide 64), Redis is also what lets it serve a limited key: it counts in the same buckets, so the key keeps one budget. Without Redis, the data plane leaves limited keys to Python.

Either way this protects the server from a runaway client; it isn't billing-grade metering.

Next steps

- Give an external partner access to one query - a partner key is the usual place for a per-key limit.
- Restrict a key to specific source IPs.
- Create a role to give a class of keys the same limit.